Building an AI Engine Optimization (AEO) System for Measuring How Well LLMs Answer Snowflake Developer Questions

Chanin Nantasenamat[1]

1 Snowflake, Developer Relations

Summary

AI coding assistants are now part of the Snowflake developer workflow, but there is no systematic way to measure whether those assistants give developers correct, current answers. We built an AEO benchmark that evaluates AI answer quality across 50 Snowflake developer questions spanning 13 product categories. Using a 2^4 factorial experiment design, we tested 16 combinations of four augmentation factors (domain prompt, citation instruction, agentic tools, self-critique) to isolate what actually improves answer quality. The best configuration (citation + agentic tools, no domain prompt) scored 93.8%, a 33 percentage-point improvement over the bare LLM baseline of 60.9%. Agentic tool access was the dominant factor (+12pp average), while self-critique was consistently counterproductive (−3pp average). These findings directly inform how Snowflake should configure its AI-powered developer tools.

1 Introduction

When a developer asks an AI assistant "How do I create a Cortex Search Service?", the quality of the answer depends on more than the model's training data. It depends on whether the assistant can search current documentation, whether it has domain-specific prompting, and whether it applies post-generation review. But which of these augmentation strategies actually matter, and how much?

In early 2026, Vercel built an AI Engine Optimization (AEO) system to track how AI coding agents reference and recommend their products. Their system measures brand visibility: does the agent mention Vercel products? Our system asks a different question: does the agent give the *right* answer?

We designed a benchmark that goes beyond brand tracking to measure multi-dimensional answer quality (correctness, completeness, recency, citation, and recommendation) against expert-authored canonical answers. Rather than testing multiple competing agents, we tested multiple augmentation configurations on a single model (Claude Opus 4.6) to isolate the effect of each deployment lever. The result is a data-backed framework for configuring AI developer tools on the Snowflake platform.

2 Methodology

2.1 Question Bank

We authored 50 questions across 13 Snowflake product categories (Cortex AI Functions, Cortex Search, Cortex Agents, Dynamic Tables, Snowpark, Streamlit in Snowflake,

Apache Iceberg Tables, Snowflake ML, SPCS, Native Apps, Streams/Tasks/Snowpipe, Governance/Security, and Architecture/Fundamentals). Each question falls into one of four test types: Explain (12), Implement (23), Debug (5), or Compare (10). For every question, we wrote a canonical answer grounded in official Snowflake documentation and defined four must-have factual elements (200 total binary pass/fail checks).

2.2 Scoring Rubric

Each response was scored on five dimensions using a 0/1/2 scale (miss/partial/full):

- **Correctness**: factually accurate per current Snowflake docs
- **Completeness**: covers all key concepts and steps
- **Recency**: uses current syntax and feature names
- **Citation**: references or directs to Snowflake documentation
- **Recommendation**: recommends the Snowflake-native approach

Maximum score per question: 10 points. Maximum per run: 500 points. Each question also has four must-have binary checks, producing a separate must-have (MH) pass rate.

2.3 Judge Panel

Every response was scored independently by three LLM judges: openai-gpt-5.4, claude-opus-4-6, and llama4-maverick. The final score for each response is the panel average. This design mitigates single-model scoring bias.

2.4 2^4 Factorial Experiment

We tested four binary augmentation factors in all 16 possible combinations:

Factor	OFF	ON
Domain Prompt	No system message	1,800-token Snowflake product knowledge primer system message
Citation	Raw question only	"Reference official Snowflake documentation" append to question
Agentic Tools	Single `CORTEX.COMPLETE` API call (parametric knowledge only)	Cortex Code subagent with web search, doc sear skills, and file I/O
Self-Critique	Single-turn generation	Two-turn generate-then-revise pattern

Non-agentic runs (8 of 16) used `SNOWFLAKE.CORTEX.COMPLETE('claude-opus-4-6', ...)` with the conversation format. Agentic runs (8 of 16) launched five parallel Cortex Code subagents, each handling a 10-question batch with full tool access. All 16 runs used Claude Opus 4.6 exclusively as the respondent model to avoid cross-model contamination.

Subagents were sandboxed to prevent access to canonical answers, scoring rubrics, or benchmark files.

3 Results

3.1 Overall Rankings

The 16 configurations produced scores ranging from 56.9% to 93.8%:

Rank	Domain	Citation	Agentic	SC	Score	MH
1		✓	✓		**93.8%**	91.5%
2		✓	✓	✓	93.2%	93.5%
3	✓	✓	✓		76.0%	90.5%
4	✓	✓	✓	✓	73.0%	91.2%
5			✓		72.2%	93.5%
6	✓				71.5%	63.5%
7	✓	✓			71.5%	54.5%
8	✓		✓	✓	71.2%	88.7%
9		✓			71.1%	48.5%
10			✓	✓	70.8%	88.2%
11	✓		✓		70.4%	89.8%
12	✓	✓		✓	67.4%	69.3%
13		✓		✓	65.7%	60.7%
14	✓			✓	62.0%	69.0%
15					60.9%	68.5%
16				✓	56.9%	66.5%

The engine split is clear: Cortex Code subagents (with tool access) averaged 77.6% score and 90.9% MH, compared to 65.9% score and 62.6% MH for single `CORTEX.COMPLETE` calls.

3.2 Main Effects

The factorial design lets us compute the average impact of turning each factor ON across all 8 paired comparisons:

Factor	Score Effect	MH Effect
Agentic Tools	**+11.7pp**	**+28.3pp**
Citation Instruction	+9.5pp	−3.5pp
Domain Prompt	−2.7pp	+0.7pp
Self-Critique	−3.4pp	+3.4pp

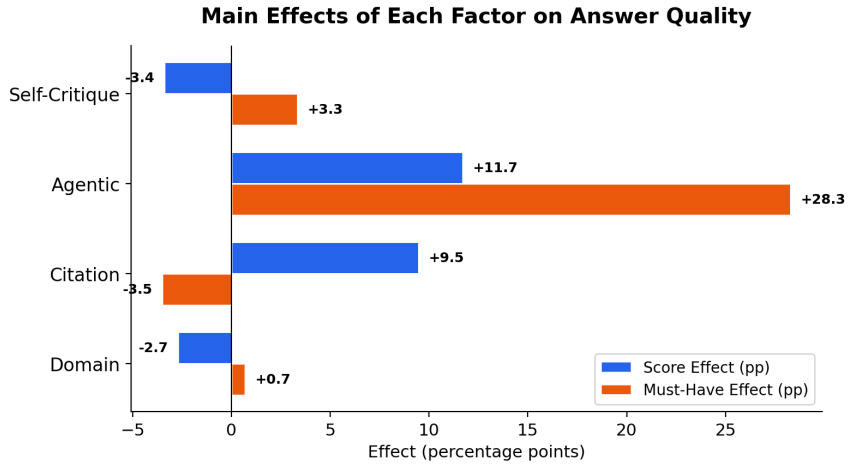

Figure 1. Main effects bar chart. Agentic tools dominate both score and must-have compliance. Self-Critique and Domain Prompt show negative score effects on average.

Agentic tools are the dominant factor. They improve both score and must-have compliance in every single paired comparison. The ability to search current documentation and invoke specialized skills is more valuable than any prompting strategy.

Citation helps score but through an indirect mechanism. The +9.5pp score effect comes largely from the Citation dimension itself (which goes from near-zero to near-perfect when citation is instructed). The biggest citation gains (+22pp) occur when combined with agentic tools, where the agent can actually look up and link to real documentation URLs.

Domain prompt has a split personality. In non-agentic conditions, it helps (+3.7pp average) by giving the model Snowflake-specific context it otherwise lacks. In agentic conditions, it actively harms performance (−9.9pp average). The worst damage occurs when all three other factors are present: adding the domain prompt to Citation + Agentic drops the score by 17–20 percentage points. The likely explanation is that the 1,800-token system prompt constrains how the agent uses its retrieved information, overriding what it finds in current docs with potentially outdated or overly rigid framing.

Self-critique is counterproductive. It hurts score in 7 of 8 paired comparisons (−3.4pp average). The two-turn "generate then revise" pattern causes the model to second-guess correct content, introduce hedging, and sometimes remove accurate details that were present in the first pass.

3.3 Category Performance

The table below compares the baseline (Run 1, no factors) against the best configuration (Run 4, Citation + Agentic) across all 13 product categories:

Category	Baseline	Best (R4)	Delta
Snowpark Container Services	67.5%	100.0%	+32.5pp
Streams, Tasks, Snowpipe	75.6%	100.0%	+24.4pp
Snowpark	49.3%	99.3%	+50.0pp
Native Apps Framework	43.3%	98.4%	+55.1pp
Architecture and Fundamentals	54.4%	97.8%	+43.4pp
Dynamic Tables	69.3%	96.7%	+27.4pp
Apache Iceberg Tables	67.4%	96.7%	+29.3pp
Governance and Security	73.4%	96.6%	+23.2pp
Cortex Search (RAG)	70.0%	94.1%	+24.1pp
Cortex AI Functions	41.3%	91.3%	+50.0pp
Snowflake ML	51.3%	90.0%	+38.7pp
Streamlit in Snowflake	76.6%	88.3%	+11.7pp
Cortex Agents	57.7%	68.9%	+11.2pp

Every category improved. The largest gains occurred in Native Apps Framework (+55.1pp), Snowpark (+50.0pp), and Cortex AI Functions (+50.0pp), where the baseline model lacked current documentation knowledge that agentic tool access provided. The weakest category remained Cortex Agents (68.9%), reflecting the relative newness of this feature and limited training data even in current documentation.

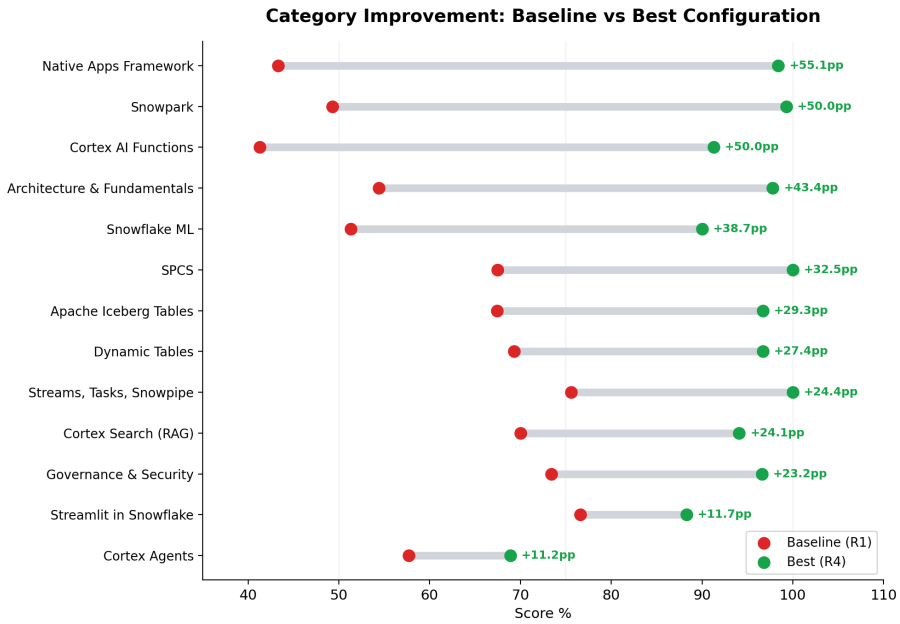

Figure 2. Dumbbell chart showing per-category improvement from baseline (Run 1) to best configuration (Run 4, Citation + Agentic). Categories sorted by delta. Every category improved, with Native Apps Framework gaining the most (+55.1pp).

3.4 Scoring Dimension Analysis

Citation is the dimension most sensitive to configuration. Without explicit citation instruction, models score near zero on the Citation dimension (0.01/2.0 in the baseline). With citation instruction plus agentic tools, it reaches 1.99/2.0. The other four dimensions (Correctness, Completeness, Recency, Recommendation) are more stable, with agentic access providing consistent 5–15% lifts across all of them.

3.5 Full Factorial Heatmap

The heatmap below shows all 16 runs against all 13 product categories, with rows grouped by whether the Agentic factor is ON (upper half) or OFF (lower half). The color gradient makes the agentic divide immediately visible: the upper half is predominantly green (high scores), while the lower half shifts toward yellow and red.

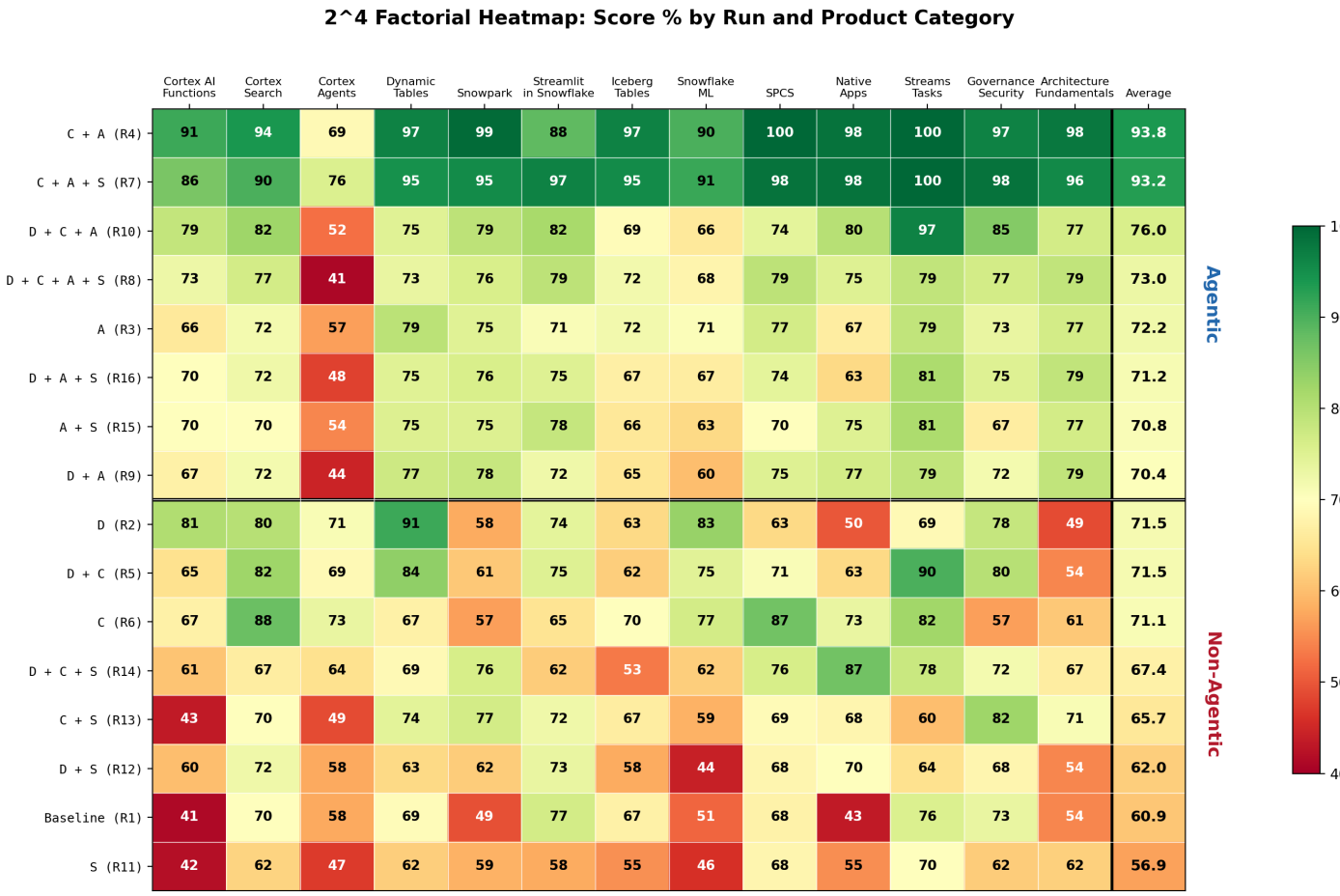

Figure 3. Category-level heatmap of all 16 factorial runs. Rows are grouped by Agentic factor (upper = ON, lower = OFF) and sorted by average score within each group. The rightmost column shows the run average. Factor abbreviations: D = Domain Prompt, C = Citation, A = Agentic, S = Self-Critique.

4 Conclusion

Four actionable findings emerge from this benchmark:

1. **Deploy agentic tools, not bigger prompts.** Access to current documentation and specialized skills produces larger quality improvements than any prompting strategy. The optimal configuration uses citation instruction and agentic tools with no domain prompt.

Beyond the main effects, two-way interaction effects reveal that factors do not act independently:

2. **Pair citation with agentic tools, not with static prompts.** Citation and agentic tools amplify each other (+6.7pp interaction effect). An agent instructed to cite sources will actively search for and link real documentation, while a non-agentic model instructed to cite can only fabricate or vaguely reference URLs.

3. **Remove the domain prompt from agentic configurations.** A static knowledge primer actively interferes with agentic tool use (−14.3pp interaction effect). When all

three other factors are present, adding the domain prompt drops the score by 17–20 percentage points. The domain prompt is only useful for non-agentic, single-call scenarios where the model has no other source of Snowflake-specific context.

4. **Do not add self-critique steps.** The generate-then-revise pattern degrades answer quality and adds latency for no benefit. If revision is desired, it should be done by a separate evaluator, not the same model reviewing its own output.

For product teams configuring Snowflake AI developer tools, the prescription is straightforward: give the agent tool access, instruct it to cite sources, and stay out of its way.

Limitations

This benchmark has several limitations worth noting:

- **Single respondent model.** All 16 runs use Claude Opus 4.6; results may differ for other models.
- **Question bank coverage.** The 50-question bank spans 13 categories but does not cover every Snowflake feature.
- **LLM-as-judge scoring.** Even with a 3-model panel, LLM judges may differ from human expert evaluation on nuanced questions. The must-have elements are binary checks that do not capture partial credit for closely related facts.
- **No TruLens integration in production scoring.** Although we built a TruLens integration (instrumented app, custom feedback functions, Snowflake connector), the 16-run factorial experiment used our custom 3-judge pipeline rather than TruLens. This means we lack standardized OpenTelemetry tracing of retrieval and generation spans, which would provide deeper observability into why agentic runs perform better (for example, which tool calls and documentation lookups contributed most to correct answers). The custom pipeline also does not produce the RAG Triad metrics (groundedness, answer relevance, context relevance) that would enable direct comparison with other TruLens-evaluated systems. Migrating the scoring pipeline to TruLens would unify evaluation with Snowflake AI Observability and make results visible in Snowsight under AI & ML > Cortex AI > Evaluations.

Next Steps

The immediate priorities are automating the pipeline for scheduled runs (to detect regressions as models and documentation change), expanding to multi-agent testing (Claude Code, Codex, Cursor) to understand competitive positioning, and building a PM-facing Streamlit interface for self-serve prompt configuration testing.

References

- Dodds, E. and Zhou, A. (2026). "How we built AEO tracking for coding agents." *Vercel Engineering Blog*. February 9, 2026.
- Snowflake Documentation. https://docs.snowflake.com

April 8, 2026